



PES University, Bengaluru
(Established under Karnataka Act 16 of 2013)

END SEMESTER ASSESSMENT (ESA) - Dec 2024

UE21CS341B - Compiler Design

Total Marks : 100.0

1.a. Explain the different phases of a compiler with a neat diagram and give an example for each phase. (8.0 Marks)

1.b. What is input buffering and why is it necessary during lexical analysis? Explain briefly the advantage of having sentinels at the end of each buffer halves in buffer pairs. (5.0 Marks)

1.c. Construct a transition diagram for:
1. Relational Operators
2. Identifiers
3. White Spaces

(6.0 Marks)

1.d. Answer the following for the given grammar:

$S \rightarrow W$
 $W \rightarrow ZXY \mid XY$
 $Y \rightarrow c \mid \epsilon$
 $Z \rightarrow a \mid d$
 $X \rightarrow Xb \mid \epsilon$

1. Construct first and follow table
2. Construct LL(1) table and Mention whether the grammar is in LL(1) or not

(6.0 Marks)

2.a. Consider the following grammar:

$E \rightarrow BB$
 $B \rightarrow cB \mid d$

1. Construct LR(0) automata.
2. Construct LR(0) parsing table. Is the grammar LR(0).
3. Show how to parse the input string "**ccdd**" using LR(0) parser.

(8.0 Marks)

2.b. Compare all table-driven bottom-up parsers based on their number of states, reduce move placement and parsing power. (4.0 Marks)

2.c. i) Define Synthesized Attributes with suitable examples.
ii) Evaluate the following Syntax Directed Definition (SDD) for the input $3 + 4 * 5$.

Production	Semantic Rule
$E \rightarrow E1 + T$	$\{ E.val = E1.val + T.val \}$
$E \rightarrow T$	$\{ E.val = T.val \}$
$T \rightarrow T1 * F$	$\{ T.val = T1.val * F.val \}$
$T \rightarrow F$	$\{ T.val = F.val \}$
$F \rightarrow num$	$\{ F.val = num.lexval \}$

1. Construct the parse tree for the input $3 + 4 * 5$
2. Construct the dependency graph
3. Determine the evaluation order of the nodes in the dependency graph and compute the final value of the expression.

(8.0 Marks)

2.d. Explain in brief Inherited Attribute and L-attributed SDT with suitable examples. (5.0 Marks)

3.a. Design an SDD (Syntax Directed Definition) to generate intermediate code for arithmetic expressions involving:

Assignment statements : $S \rightarrow id = E$

Addition : $E \rightarrow E1 + E2$

Unary minus : $E \rightarrow -E1$

Parentheses : $E \rightarrow (E1)$

Identifiers : $E \rightarrow id$

(5.0 Marks)

3.b. i) Discuss the rules for converting L-attributed SDD to L-attributed SDT.
ii) Consider the following L-attributed SDD (to generate intermediate code) for while-statement.

Production	Semantic Rule
$S \rightarrow \text{while } (C) S1$	$L1 = \text{new label}();$ $L2 = \text{new label}();$ $C.\text{true} = L2$ $C.\text{false} = S.\text{next}$ $S1.\text{next} = L1$ $S.\text{code} = \text{label}(L1) \mid \mid C.\text{code} \mid \mid \text{label}(L2) \mid \mid$ $S1.\text{code} \mid \mid \text{gen}(\text{'goto' begin})$

Convert this L-attributed SDD into an L-attributed SDT. (3.0 Marks)

3.c. Generate Three-address code for the following code
while (A < C and B > D) do
 if A = 1 then C = C + 1
 else
 while A <= D
 do A = A + B (5.0 Marks)

3.d. Write the Quadruple, Triple and Indirect Triples representation for the following Three-Address Code.

t1 = -c
t2 = b * t1
t3 = - c
t4 = b * t3
t5 = t2 + t4
a = t5 (6.0 Marks)

3.e. Construct control flow graph (CFG) for the given three-address code (TAC) and then convert the code in CFG to SSA form.

(6.0 Marks)

```
i = 1
j = 1
k = 0
L1: ifFalse k < 100 goto L3
    ifFalse j < 20 goto L2
    j = i
    k = k + 1
    goto L1
L2: j = k
    k = k + 1
    goto L1
L3: return j
```

4.a. Explain the following machine-independent code optimization techniques in detail. Provide examples for each to illustrate their working.

1. Constant Folding and Constant Propagation
2. Common Subexpression Elimination
3. Dead Code Elimination

(6.0 Marks)

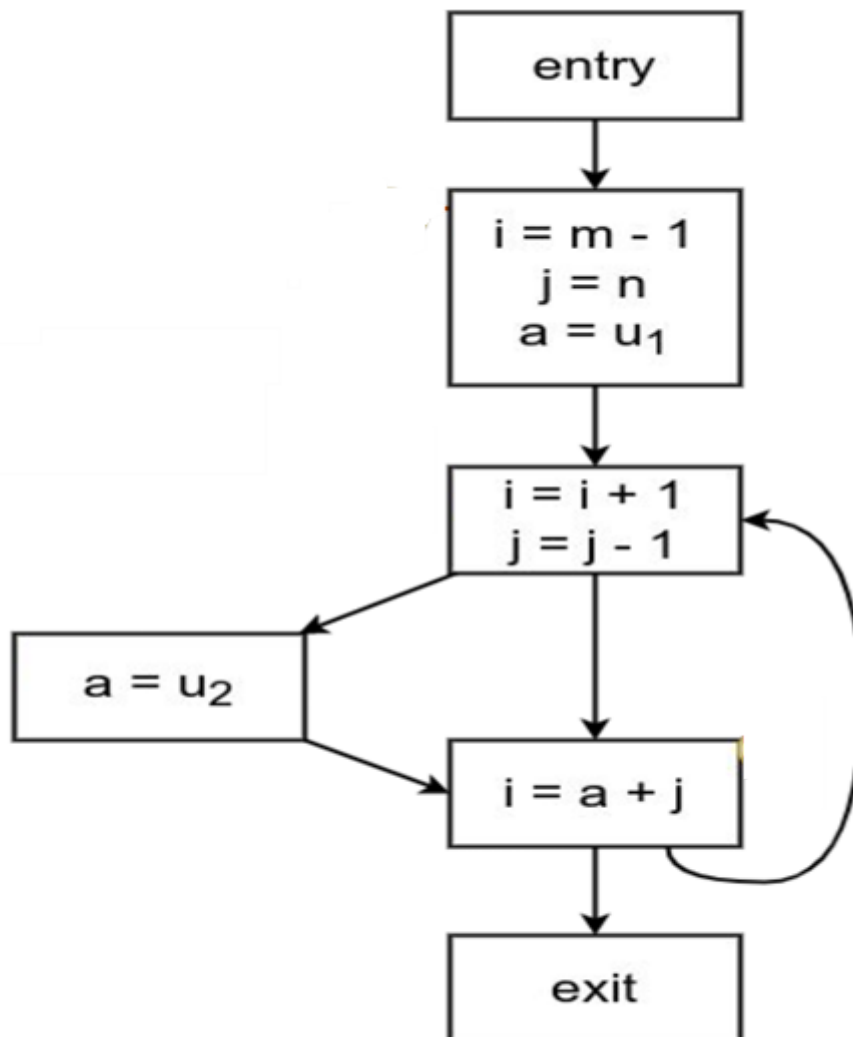
4.b. i) Explain Index addressing mode and Indirect addressing mode with examples.

ii) Generate target code sequence for the following three address statements assuming **a** and **b** are arrays whose elements are 4 byte values.

```
x = a[i]
y = b[j]
a[i] = y
b[j] = x
```

(6.0 Marks)

4.c. Perform Live-Variable Analysis on the following control flow graph. (8.0 Marks)



4.d. Explain the concepts of Calling Sequence and Return Sequence in procedure call implementation with suitable examples. (5.0 Marks)